La última transmisión — Informe del parcial

Parcial práctico de Ingeniería de Software · motor de historias interactivas

Integrantes	Esteban Ramírez Rios
Versión descrita	Motor v1.1.0 (v1.0.0 es la versión previa; v1.1.0 incorpora la solicitud de cambio de la hora 6, ver §8). Anexo: **anexo.pdf** (diagramas A, matriz de trazabilidad B, tablero C).
Entrega	14 de octubre de 2026 · Repositorio: historial en evidencia/historial_git.txt

1. Fundamentos (Unidad 1)

Problema. Un colectivo de escritores prepara una ficción interactiva para un festival universitario y no puede depender de un programador cada vez que cambia una escena o una decisión. Necesita un motor que cargue relatos desde archivos, ofrezca opciones condicionadas por lo ocurrido y avise qué corregir cuando un archivo es incorrecto. **Actores:** escritora principal (edita escenas), diseñador de la experiencia (define condiciones y efectos), coordinadora del festival (necesita diagnósticos, no cierres inesperados) y lector (juega). **Alcance:** motor local de consola, contenido en JSON, reglas de igualdad sobre booleanos, validación previa, tres finales y una segunda historia sin tocar el motor. **Fuera de alcance:** cuentas, base de datos, guardado, red, editor visual, audio, imágenes, lenguaje de expresiones, retroceso de varios niveles y rehacer.

Riesgo	Prob. / impacto	Mitigación aplicada
R1. Contenido mal escrito produce un cierre del programa o una partida sin salida.	Media / alta	Validación acumulativa antes de jugar (RF-12..17); bloqueo informado con opción de reiniciar (RF-09); pruebas de contenido mutado (500 casos, RNF-03).
R2. El cambio de la hora 6 llega con poco tiempo y rompe la coherencia entre requisitos, modelos y código.	Alta / alta	Motor genérico sin lógica por historia (RNF-01); trazabilidad por ID (anexo B); tiempo reservado para el cambio y pruebas de regresión; versión previa etiquetada en Git.

Prioridad de atributos de calidad. (1) *Modificabilidad del contenido*: es la razón de ser del producto, porque la escritora debe cambiar el relato sin tocar código. (2) *Confiabilidad*: un cierre inesperado arruina la experiencia en el festival. (3) *Claridad de los errores*: la coordinadora debe saber qué corregir. Quedan por debajo el rendimiento y la portabilidad (relatos pequeños, Python estándar).

Responsabilidad profesional. El relato es obra de sus autores: el motor no lo modifica ni lo copia, no recoge datos de los lectores (sin red ni guardado) y las historias de prueba se mantienen separadas del código. Se usan pocas dependencias, todas con licencia permisiva y declaradas (§7), y se declara el uso de herramientas de IA (§9) distinguiendo lo propio de lo adaptado.

2. Proceso (Unidad 2)

Elegido: incremental e iterativo con incrementos cortos —(1) especificación y modelos, (2) núcleo del motor con pruebas, (3) verificación y entrega, (4) incremento del cambio—. **Alternativa comparada: cascada.** Se descartó porque concentra la validación al final y absorbe mal un cambio anunciado a mitad de la ventana; el enfoque incremental deja un producto ejecutable en cada paso y hace barato el cambio de la hora 6.

Actividad	Cómo se realiza en esta entrega
Especificación	RF, RNF y criterios de aceptación escritos antes del código; supuestos registrados (§4).
Desarrollo	Capas independientes (dominio, contenido, sesión, interfaz); un commit por capa.
Validación	103 pruebas automatizadas, tres recorridos de finales, 3 RNF medidos (§7).
Evolución	Hora 6: versión previa etiquetada, análisis de impacto, cambio, trazabilidad actualizada (§8).

3. Agilidad (Unidad 3)

Backlog y tablero. 24 ítems con responsable y estado en *evidencia/backlog.csv*; tablero Kanban (Por hacer / En curso / Hecho). Las capturas inicial y actual están en el anexo C; deben complementarse con la captura de la herramienta real del equipo. **Prácticas aplicadas:** (a) *criterios de aceptación ejecutables*: CA01–CA03 de la guía son pruebas automáticas, no solo texto; (b) *integración frecuente con pruebas antes de cada commit* y revisión cruzada entre integrantes. **Alternativa descartada:** programación en pareja síncrona, inviable en una ventana asincrónica; se sustituyó por revisión cruzada.

Definición de terminado: el ítem tiene código o documento en Git; las pruebas relacionadas pasan; el requisito tiene criterio de aceptación; el otro integrante lo revisó; la trazabilidad indica dónde está la evidencia.

Aprendizaje y mejora. La primera medición de RNF-01 buscó los identificadores de las historias como texto dentro del código y marcó «no cumple» por la palabra «entrada» en dos docstrings: un falso positivo del instrumento, no del motor. *Acción:* medir con análisis de sintaxis (*ast*) solo sobre literales y nombres ejecutables, y comprobar siempre el instrumento antes de interpretar su resultado.

4. Requerimientos (Unidad 4)

Los RF son atómicos y se vinculan con las necesidades N01–N06 de la guía; cada uno se traza a modelos, componentes y evidencia en el anexo B.

ID	N	Requisito (comportamiento observable)	Criterio de aceptación
RF-01	N01	Cargar una historia desde un archivo JSON indicado por ruta.	Cargar la_ultima_transmision.json da 6 escenas; cargar el_faro.json funciona sin editar código.
RF-02	N01	La historia define la escena inicial y los valores iniciales de las variables; toda partida nueva parte de ahí.	Partida nueva: escena E1, conoce_origen y confia_en_la_voz en falso.
RF-03	N02	Mostrar el texto de la escena actual.	Al iniciar se muestra el texto de E1.
RF-04	N02	Mostrar solo las opciones cuyas condiciones se cumplen (todas las igualdades; mapa vacío = visible).	CA01: tras revisar e investigar, E3 muestra «Activar con código» y «Cortar»; sin investigar, «Activar sin verificar» y «Cortar».
RF-05	N03	Al elegir una opción habilitada, aplicar sus efectos y pasar a su destino.	Investigar en E2 deja conoce_origen en verdadero y lleva a E3.
RF-06	N03	Rechazar una opción inexistente, no habilitada o elegida en una escena final, con motivo y sin alterar escena, variables ni historial.	CA02: «activar con código» sin conocer el origen se rechaza aunque se envíe su ID; sigue en E3.
RF-07	N03	Rechazar entradas de teclado no reconocidas sin alterar la partida.	Escribir «abc» o «9» muestra «Entrada no válida» y la escena no cambia.
RF-08	N04	Recordar durante la partida el valor de cada variable y usarlo en las condiciones posteriores.	El efecto de E2 habilita una opción distinta en E3.
RF-09	N04	Si una escena no final no tiene opciones habilitadas, informar el bloqueo y permitir reiniciar.	Historia de prueba bloqueada: se muestra «[Bloqueo]» y se ofrece jugar de nuevo.
RF-10	N05	Reconocer una escena final, mostrar su texto y avisar el fin.	Los tres recorridos terminan en E4, E5 y E6 con «FIN DE LA HISTORIA».
RF-11	N05	Iniciar otra partida restaurando escena y variables iniciales y descartando el historial.	Tras un final, reiniciar vuelve a E1 con variables iniciales; no queda estado previo.
RF-12	N06	Rechazar antes de jugar un archivo inexistente, ilegible, no UTF-8, no JSON o de estructura incorrecta.	Un JSON truncado da E_ILEGIBLE con línea y columna.
RF-13	N06	Rechazar campos obligatorios ausentes (historia, escena, opción).	Quitar «titulo» o el «destino» de una opción da E_CAMPO_AUSENTE con el campo.
RF-14	N06	Rechazar inicio inexistente, IDs de escena repetidos, IDs de opción repetidos en su escena y destinos inexistentes.	CA03: destino E99 da E_DESTINO con ‘E99’ y no abre sesión.
RF-15	N06	Rechazar variables no declaradas y valores no booleanos en variables, condiciones y efectos.	Efecto con valor 1 o variable «fantasma» da E_NO_BOOL / E_VARIABLE.
RF-16	N06	Rechazar escenas finales con opciones y no finales sin ellas.	Escena final con una opción o no final vacía da E_FINAL_OPCIONES.
RF-17	N06	Informar cada rechazo con causa y campo/ID, acumulando todos los problemas, sin abrir sesión jugable.	Un archivo con dos errores informa ambos en un solo mensaje.
RF-18	N01, N06	Permitir validar sin jugar (--validar) y reproducir un recorrido por IDs (--ruta).	Códigos de salida 0, 1 y 2 según el caso.
RF-19	N03, N04	Cambio hora 6. Deshacer la última decisión restaura exactamente la escena, las variables y el historial anteriores a la última elección válida.	Desde E2, investigar lleva a E3 con conoce_origen verdadero; deshacer devuelve a E2 con conoce_origen falso.
RF-20	N03, N04	Cambio hora 6. El retroceso es de un solo nivel: tras deshacer no hay otro hasta una nueva elección válida; al comenzar y tras reiniciar no hay retroceso disponible. No se exige rehacer.	Un segundo deshacer no cambia nada y se informa; al continuar sin investigar, E3 muestra «Activar sin verificar» y «Cortar».
RF-21	N03	Cambio hora 6. Una opción inexistente o no habilitada no modifica la sesión ni sustituye la última elección válida que se deshará.	Tras una entrada inválida, deshacer revierte la elección válida anterior.
RF-22	N03, N05	Cambio hora 6. Se puede deshacer desde un final: se vuelve a la escena desde la que se eligió y se restauran los valores previos.	Desde E4, E5 o E6, deshacer regresa a E3 con las variables previas a esa elección.

Requisitos no funcionales medibles

ID	Atributo	Requisito medible	Procedimiento (reproducible)	Resultado
RNF-01	Modificabilidad	Incorporar una historia nueva de ≥ 3 escenas sin modificar ningún archivo del motor; 0 identificadores de historia en su código.	pruebas/verificar_rnf.py: hash SHA-256 de transmision/*.py antes y después de cargar y jugar el_faro.json; análisis ast de literales y nombres.	0 archivos modificados; 0 apariciones; 4 escenas, final alcanzado.
RNF-02	Claridad de errores	10 de 10 variantes de error informan causa (código) y campo o ID involucrado.	Mismo script: genera una variante por categoría desde el JSON real y comprueba el mensaje.	10/10.
RNF-03	Confiabilidad	0 excepciones no controladas ante contenido o entradas inválidas.	Mismo script, semilla 20261014: 500 contenidos mutados y 500 partidas con entradas aleatorias al motor y a la consola.	0 excepciones.

Conflicto del cliente y acuerdo provisional

La escritora quiere poder retroceder siempre; el diseñador quiere decisiones irreversibles. **Pregunta de aclaración:** ¿el retroceso debe existir siempre, solo en escenas que el relato marque como reversibles, o nunca? **Acuerdo provisional**

(v1.0.0): no implementar retroceso, sin presuponer la decisión del cliente. **Reemplazado en la hora 6:** el cliente decidió permitir deshacer solo la última decisión (RF-19 a RF-22); la pregunta de «siempre / solo en escenas reversibles / nunca» queda resuelta como «un nivel, en cualquier escena».

Preguntas y supuestos

- **P1.** ¿Son obligatorios «condiciones» y «efectos» aunque estén vacíos? *Supuesto:* sí, el contrato los lista como campos de la opción; si faltan se informa E_CAMPO_AUSENTE.
- **P2.** ¿El lector elige por número o por ID? *Supuesto:* ve opciones numeradas; el motor acepta también el ID y rechaza el de una opción no habilitada (CA02).
- **P3.** ¿Se muestran las variables al lector? *Supuesto:* no; el estado es interno y solo se ve por sus consecuencias.
- **P4.** ¿Debe detectarse un final inalcanzable o un ciclo? *Respuesta de la guía:* no se exige; solo se informa el bloqueo durante la partida.

Defecto detectado en la revisión y su corrección

Revisor: **[COMPLETAR]**. El borrador de RF-04 decía «en E3 deben verse exactamente dos opciones»: ataba el requisito a una escena de la historia y contradecía la exigencia de que el motor no tenga decisiones específicas de E1, E2 u otros IDs. **Corrección:** RF-04 se redactó como regla general (mostrar solo opciones habilitadas) y el caso de E3 pasó a ser su criterio de aceptación CA01.

5. Modelado (Unidad 5)

Modelo	Figura	Qué representa
Contexto	A.1	Escritora, diseñador, coordinadora y lector alrededor del motor y los archivos JSON.
Casos de uso	A.2	CU1 cargar, CU2 validar, CU3 ver escena, CU4 elegir, CU5 reiniciar, CU6 reproducir recorrido.
Clases del dominio	A.3	Historia, Escena, Opción, Sesión, Diagnóstico y las dos excepciones del motor.
Secuencia: elegir una opción	A.4	Consola, Sesión, reglas e Historia, con la rama de opción no habilitada (estado intacto).
Estados de una sesión	A.5	EnCurso, Bloqueada y Finalizada; reiniciar vuelve a EnCurso.

Los modelos usan el mismo vocabulario que los requisitos y el código (historia, escena, opción, condición, efecto, variable, sesión, diagnóstico). El mapa narrativo (A.7) ayuda a leer la historia, pero no sustituye estos modelos.

6. Arquitectura (Unidad 6)

Arquitectura en capas con dependencias en un solo sentido (anexo A.6): la **interfaz** (consola) usa la **sesión** y el **cargador**; la sesión usa las **reglas** y el **modelo**; el **contenido** (cargador, validador, diagnóstico) produce el modelo a partir de los archivos. El dominio no conoce ni la consola ni el disco.

Componente	Responsabilidad
consola	Entrada y salida, menús, reinicio, códigos de salida. Sin reglas del juego.
sesion	Escena actual, variables, historial; elegir, reiniciar, detectar bloqueo; no modifica la historia.
reglas / modelos	Evaluar condiciones, aplicar efectos (funciones puras); estructuras inmutables de la historia.
cargador / validador / diagnostico	Leer el archivo, validar todos los casos de la guía y construir el modelo o informar diagnósticos.
historias/*.json	Datos de los escritores; se editan sin tocar código.

Decisión D1 — contenido declarativo en JSON, validado antes de jugar

Contexto: la escritora no programa y el diseñador necesita condiciones sobre lo ocurrido. **Alternativas:** (a) historias escritas como código Python, (b) YAML, (c) reglas fijas en el motor. **Elección:** JSON, que la guía propone y la biblioteca estándar lee. **Consecuencias:** +modificabilidad y validación centralizada; −JSON no admite comentarios y es sensible a comas, lo que se compensa con errores de sintaxis con línea y columna (RF-12).

Decisión D2 — estado de la partida separado de un modelo inmutable

Contexto: reiniciar debe borrar todo rastro y un error no debe dejar la partida a medias. **Alternativas:** (a) mutar la historia durante el juego, (b) estado global, (c) registro de eventos. **Elección:** Historia inmutable; Sesión copia las variables, calcula los efectos sobre una copia y confirma el cambio al final. **Consecuencias:** reinicio trivial y elecciones atómicas (U3, U5); a cambio hay copias pequeñas de un diccionario, sin costo apreciable.

Separación contenido–reglas–interfaz demostrada: la segunda historia (el_faro.json, 4 escenas, variables y IDs distintos) se carga y se juega con el mismo código; RNF-01 confirma 0 archivos del motor modificados y 0 identificadores de historia en su código.

7. Implementación y verificación (Unidad 7)

Decisión de diseño detallado. Las reglas son funciones puras en un módulo propio (*reglas.py*), y la consola recibe *leer* y *escribir* como parámetros. Así la lógica se prueba sin teclado ni pantalla, el acoplamiento entre interfaz y dominio es mínimo y cualquier otra interfaz (web, escritorio) podría reutilizar sesión y reglas.

Biblioteca	Uso	Licencia
Biblioteca estándar de Python (json, argparse, dataclasses)	Motor, validación y consola	PSF License

Biblioteca	Uso	Licencia
pytest	Pruebas automatizadas	MIT
reportlab, matplotlib, graphviz (solo documentación, no son del producto)	Generar informe, anexo y diagramas	BSD, matplotlib License (BSD-style), EPL

Caso	Qué comprueba	Dónde
U1	Condiciones cumplidas y no cumplidas habilitan solo las opciones correspondientes.	pruebas/test_u1_condiciones.py
U2	Elegir aplica efectos y avanza al destino correcto.	pruebas/test_sesion.py
U3	Opción inexistente o no habilitada deja el estado intacto.	pruebas/test_sesion.py
U4	Errores de contenido: 22 mutaciones del JSON real, 6 archivos ilegibles, codificación, acumulación, CA03.	pruebas/test_u4_validacion.py
U5	Reiniciar restaura escena y variables iniciales y elimina el historial de retroceso (extendida en v1.1.0).	pruebas/test_sesion.py
U6	Deshacer restaura escena y variables, consume el retroceso; entrada inválida conserva el historial; nueva elección lo rehabilita; desde un final vuelve a E3.	pruebas/test_u6_deshacer.py
I1	Archivo real + motor + consola hasta los tres finales; segunda historia; CA01–CA03; deshacer desde un final (v1.1.0).	pruebas/test_i1_integracion.py
RNF	Versión reducida de los tres RNF.	pruebas/test_rnf.py

Resultados reales: 103 pruebas pasan, ninguna falla (resultados/pruebas.txt); RNF-01, RNF-02 y RNF-03 cumplen (resultados/rnf.txt). **Fallos pendientes:** ninguno conocido en la versión 1.1.0. **Limitaciones:** no se analizan todos los caminos del relato; el retroceso es de un solo nivel y no hay rehacer, como pide el cliente.

8. Gestión del cambio (hora 6)

Versión previa: motor v1.0.0 (78 pruebas, sin retroceso; acuerdo provisional «no implementar retroceso»). **Versión posterior:** motor v1.1.0 (103 pruebas, 3/3 RNF). Procedimiento y análisis completos en docs/gestion_cambio.md.

Solicitud. El colectivo decidió que el lector pueda deshacer su última decisión, revirtiendo sus consecuencias: restauración exacta de escena y variables, un solo nivel (sin rehacer), sin retroceso al comenzar ni tras reiniciar, entrada inválida que no altera el historial y deshacer desde E4, E5 o E6 hacia E3. Reemplaza el acuerdo provisional de no implementar retroceso.

Artefacto	Impacto
Requisitos	Nuevos RF-19 a RF-22 (N03–N05); RF-11 se mantiene y se verifica también para el historial de retroceso; acuerdo provisional reemplazado.
Modelos	Clases: Sesion.deshacer(), puede_deshacer y SinRetroceso (A.3). Estados: transición deshacer() (A.5). Secuencia: guarda el estado previo al elegir (A.4). Contexto, casos de uso y componentes sin cambio estructural.
Código	sesion.py (instantánea previa solo tras validar la elección; deshacer la restaura y consume), consola.py (comando «u» y «@deshacer» en --ruta), __init__.py (versión 1.1.0). Reglas, validador, cargador, modelos e historias sin cambios.
Pruebas	Nueva U6 (pruebas/test_u6_deshacer.py); U5 extendida (reiniciar elimina el historial); I1 extendida (deshacer desde un final tras cargar el archivo y por consola). Mínimo final: 6 unitarios + 1 de integración.
Backlog y trazabilidad	B21–B23 en «Hecho»; matriz B del anexo con RF-19 a RF-22 y U6.

Qué lo demuestra: pruebas/test_u6_deshacer.py, test_sesion.py::test_u5_reiniciar_elimina_el_historial..., test_i1_integracion.py::test_i1_deshacer_desde_un_final...; 103 pruebas pasan (resultados/pruebas.txt); RNF-01 sigue cumpliendo (el motor no incorpora IDs de la historia); demostración: *python -m transmision --ruta revisar,investigar,@deshacer,continuar*.

9. Fuentes, herramientas de apoyo y aportes

Herramientas de IA. Se utilizó Claude (Anthropic) para generar un borrador del código, las pruebas, los diagramas y este informe a partir de la guía del parcial. **Verificación:** se ejecutaron las pruebas y la verificación de RNF (103 pruebas; 3/3 RNF), se revisaron los diagramas y se corrigieron errores hallados (p. ej., un falso positivo de medición de RNF-01 y rombos de composición invertidos). **[COMPLETAR]** Cada integrante debe confirmar que revisó y entiende el código y que este uso de IA está permitido por las indicaciones del docente. No se reutilizaron fragmentos de tutoriales.

Aportes por integrante. Esteban Ramírez Rios: análisis, requisitos, diagramas, implementación del motor, pruebas, verificación de RNF, documentación y evidencia (todas las tareas B01–B18 del tablero; ver historial_git.txt).